

Improve your feature flagging with AWS AppConfig Version Labels

by Steve Rice and Ivan Garcia | on 09 OCT 2023 | in [Best Practices](#), [Developer Tools](#), [Management Tools](#), [Technical How-to](#) | [Permalink](#) | [Share](#)

[AWS AppConfig](#) is a feature of [AWS Systems Manager](#) that helps you increase your software deployment velocity and confidence. Providing dynamic configuration and feature flagging, AppConfig is a tool that decouples your feature releases from code deployments. You can create new features and push the code to production with the new feature hidden behind a feature flag. Once you are ready for users to try out the new feature you can toggle the feature flag to turn it on, perhaps at first limited to only internal testers, then beta users, and then finally to all users. On launch day you launch by deploying a configuration change rather than a code change and if something goes wrong you can rollback that change instantly. At Amazon we call the practice of using feature flags and dynamic configuration, [Continuous Configuration](#).

In this blog post, you will learn how to use a powerful new capability called **AWS AppConfig Version Labels**. This allows you to manage your configuration and feature flag data in many structured ways, and can simplify your workflow across multiple dimensions. Version Labels supplement the configuration version numbers that are generated by AppConfig with your own identifier allowing you to more easily track changes in configuration data as it flows through AppConfig and ultimately gets consumed by your application.

What problem does AWS AppConfig Version Labels solve?

First, a bit of context. Our recommended way to use AppConfig is to create a Configuration Profile the uses Hosted Configuration Versions which lets you store and edit configuration data directly within AppConfig as opposed to having AppConfig pull the data from a separate service like S3 at deployment time. When using [Hosted Configuration Versions](#), every version is immutable so every time you go to change your configuration data you will end up saving a new version and AppConfig will automatically assign a version number to it — version 1, version 2, and so on. Those version numbers are useful because later on when you go to make a deployment you will need to tell AppConfig exactly which version you want to deploy. However, there are some cases where it's useful to provide your own additional identifier for each version and that's where [Version Labels](#) come in.

Example use-cases for Version Labels

You can use Version Labels as a shorthand user-friendly identifier for each version. For example, in the screenshot below we have four versions of our configuration each with a version label that ties that version to a major release of an application. That way years later it's easy to travel back in time and see what the configuration looked like for any given release.

Hosted configuration versions

Version	
4	1.0_Public ▲
🔍 Search for a version.	
4	1.0_Public ✓
3	1.0_Preview
2	MVP_Release
1	initial

Figure 1. Display of Version Labels next to version numbers in the AWS Management Console

If you're regularly importing configuration data into AppConfig from some other versioned source of truth, such as a git repository, version labels let you associate the versions you're creating in AppConfig to the equivalent version in the source system. For example, you can provide a git commit ID or tag as a version label. That way it's easy to trace configuration changes all the way back to their source.

Similarly, if you're using AppConfig in multiple AWS Regions and are trying to deploy the same configuration data to different regions, you might run into the problem where the Version Numbers in different regions might not match. For example, say you had one Configuration Profile in us-east-1, and then a different one in us-west-2, and some bit of automation that regularly publishes your configuration data to both regions. As long as both regions have the same number of versions, the version numbers would naturally match each other. But what if you then manually created a version in one of the regions but not the other? Then the next time your automation runs it might be creating version 3 in one region, but version 4 in the other, even though to you they're the same version. Version Labels make this easier to handle by letting your automation supply the same label to both versions so that it's easy to see which versions correspond to each other.

How can I use Version Labels?

Several of AppConfig's APIs and web console screens have been extended with Version Labels support. They are organized below.

Creating

The [CreateHostedConfigurationVersion](#) API and the web console screens for creating Hosted Configuration Versions (both Feature Flag and Freeform) have all been extended to let you supply a Version Label when you're creating a new version.

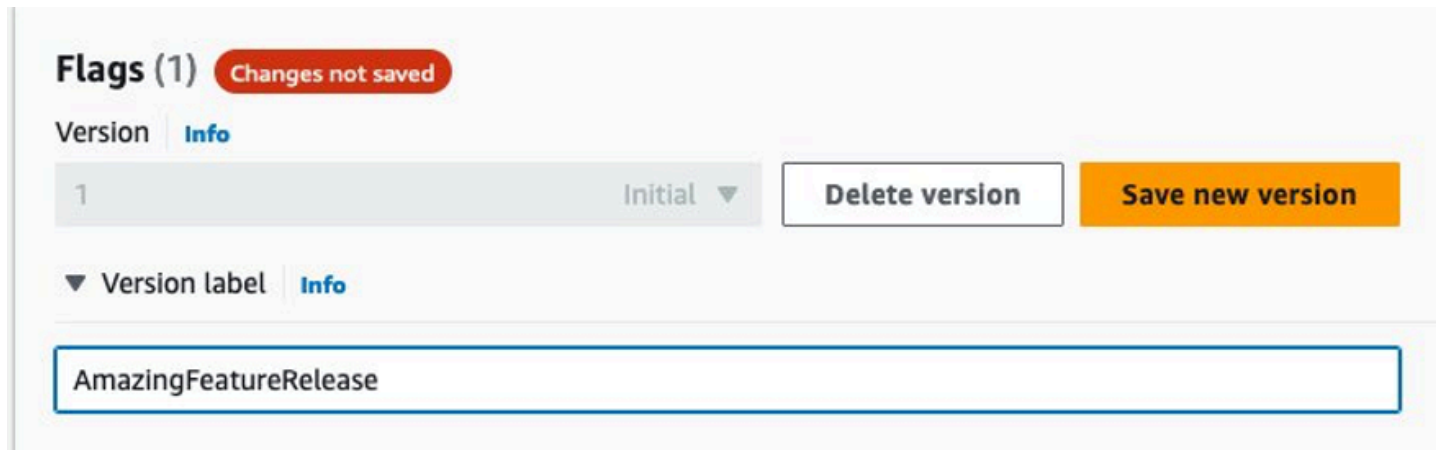


Figure 2. Adding a new version of a Configuration Profile and where to add the Version Label information

Below is a Bash shell example for how you can use the AWS CLI within a git repository to create a new Hosted Configuration Version with the current contents of a given file and including a Version Label with the latest git commit's short hash.

```
aws appconfig create-hosted-configuration-version \
  --application-id <YourAppId> \
  --configuration-profile-id <YourProfileId> \
  --content-type "text/plain"
  --content fileb://configuration.txt --version-label $(git rev-parse --short HEAD) -
```

Consuming and Debugging

[GetLatestConfiguration](#) is the main API used by applications to retrieve configuration data from AppConfig and it now includes a Version Label field in its responses if the deployed configuration had one populated. We highly recommend logging these Version Labels in whatever code you have that call `GetLatestConfiguration`. Having these labels in your logs is a powerful debugging aid because it makes it much easier to track which version of a configuration your application had loaded at a given point in time.

Deploying

When starting deployments, you can select the version to deploy based on its version label, which makes it easier for operators to choose the right version to deploy.

Start deployment

Deployment details

Use the options on this page to deploy a new or updated application configuration. [Learn more](#)

Environment

Choose an environment to deploy to.

Prod ▼

or

Create environment

Hosted configuration version

Choose a hosted configuration version to deploy.

4

1.0_Public ▼

Deployment strategy

Choose a deployment strategy to deploy your configuration data slowly (recommended) or quickly. [Learn more](#)

Select a deployment strategy ▼

or

Create deployment strategy

Deployment description - Optional

Enter a description for this deployment.

► Additional encryption options [New](#)

► Tags

Cancel

Start deployment

Figure 3. Deploying a Configuration Profile. In the version dropdown, you can see the Version Label you have added next to the version number

Even if you're kicking off deployments programmatically the [StartDeployment](#) API lets you supply a Version Label in place of a Version Number:

```
aws appconfig start-deployment \  
--application-id <YourAppId> \  
--configuration-profile-id <YourProfileId> \  
--environment-id <YourEnvId> \  
--deployment-strategy-id 'AppConfig.Linear50PercentEvery30Seconds' \  
--configuration-version '1.0_Preview'
```

Finding and Filtering



The [ListHostedConfigurationVersions](#) API has a parameter that lets you filter versions by their Version Label, either by exact match or by prefix if you include a wildcard.

```
aws appconfig list-hosted-configuration-versions \
--application-id <YourAppId> \
--configuration-profile-id <YourProfileId> \
--version-label '1.0*'
```

```
{
  "Items": [
    {
      "ApplicationId": "rd7sjl4",
      "ConfigurationProfileId": "dim47hi",
      "VersionNumber": 4,
      "ContentType": "application/json",
      "VersionLabel": "1.0_Public"
    },
    {
      "ApplicationId": "rd7sjl4",
      "ConfigurationProfileId": "dim47hi",
      "VersionNumber": 3,
      "ContentType": "application/json",
      "VersionLabel": "1.0_Preview"
    }
  ]
}
```

Pointers, Limitations, and Caveats

There are an additional items worth highlighting about Version Labels:

- Version Labels are optional. You can have a Configuration Profile where some versions have a Version Label, while others do not.
- Version Labels must be unique across all Hosted Configuration Versions in a given Configuration Profile. They do not need to be unique across different Configuration Profiles.
- Version Labels, and Hosted Configuration Versions as a whole, are immutable. So you can only specify a Version Label when you're creating a new version. You can't add, remove, or change the Version Label afterwards unless you delete and recreate the entire Hosted Configuration Version.
- Version Labels are free form text fields. Spaces are allowed, as are special characters, with the exception of asterisks (*) which are not permitted.
- Version Labels only apply to Hosted Configuration Versions, not to other types of Configuration Profiles such as S3 source profiles.

Conclusion

Using AWS AppConfig Version Labels opens the door to simplified workflows when managing feature flag and dynamic configuration data. The ability to set and reference Version Labels allows for cross-account, cross-region, and cross-source synchronization of configuration data. Using Version Labels also provides additional safeguards to give you better understanding and visibility into your own configuration data, which makes using feature flags safer.

You can read more about AWS AppConfig in the links below:

- [Using AWS AppConfig Feature Flags](#) blog post
- [AWS AppConfig Feature Flag](#) workshop
- [What is AWS AppConfig?](#) documentation
- [Continuous Configuration at the Speed of Sound](#) blog post

TAGS: [AWS AppConfig](#), [Dynamic Configuration](#), [feature flags](#)

